

Python — La visualisation de données

Document 20 — Pandas (.plot)

Série « Visualisation » · explorer vite depuis un DataFrame

Table des matières

- 1. Pourquoi tracer depuis un DataFrame
- 2. Première image : `df.plot()` en une ligne
 - 2.1 `Series.plot` et `DataFrame.plot`
 - 2.2 Le lien avec Matplotlib : `.plot()` renvoie un Axes
- 3. Un graphique complet
- 4. Les types de tracés
- 5. Plusieurs graphiques : `subplots` et `ax=`
 - 5.1 `subplots=True` : une grille automatique
 - 5.2 `ax=` : superposer sur un graphique existant
- 6. Styles et personnalisation
- 7. Les outils de `pandas.plotting`
- 8. Sauvegarde et export
- Récapitulatif
- Questions de vérification

1. Pourquoi tracer depuis un DataFrame

Pandas ne réinvente pas la visualisation : sa méthode `.plot()` s'appuie sur Matplotlib en coulisse. Son intérêt est ailleurs — tracer directement depuis un DataFrame, en une ligne, sans extraire les colonnes à la main. C'est l'outil de l'exploration : on regarde vite à quoi ressemble la donnée avant de décider quoi en faire.

L'idée clé : là où Matplotlib demande `ax.plot(df.index, df["close"])`, Pandas se contente de `df["close"].plot()`. L'index du DataFrame devient automatiquement l'axe x — idéal pour les séries temporelles, où l'index est souvent une date.

Atout	Ce que ça change
Concision	Un graphique en une ligne, directement sur la donnée.
Index = axe x	Les dates de l'index servent d'abscisse sans réglage.
Colonnes = séries	Chaque colonne devient une courbe, avec son nom en légende.
Matplotlib dessous	On récupère un Axes : tout Matplotlib reste accessible.

Pourquoi ça compte. Pandas `.plot()` est fait pour explorer, pas pour le livrable final. On regarde vite, puis on repasse à Matplotlib (qu'il utilise de toute façon) pour peaufiner. C'est la séparation exploration / livrable du document 00.

2. Première image : `df.plot()` en une ligne

2.1 Series.plot et DataFrame.plot

Toute Series et tout DataFrame possèdent une méthode .plot(). Sur une Series, on obtient une courbe ; sur un DataFrame, une courbe par colonne, avec une légende automatique.

```
import pandas as pd

df = pd.read_csv("prix.csv", index_col="date", parse_dates=True)

df["close"].plot()          # une Series -> une courbe
df.plot()                  # tout le DataFrame -> une courbe par colonne
index_col="date" et parse_dates=True placent les dates en index : elles deviennent l'axe x, formaté proprement, sans aucun réglage supplémentaire.
```

2.2 Le lien avec Matplotlib : .plot() renvoie un Axes

Point essentiel : .plot() ne fait pas que dessiner, il renvoie un objet Axes Matplotlib. On le récupère pour continuer à le personnaliser avec les méthodes vues au document 10.

```
import matplotlib.pyplot as plt

ax = df["close"].plot()      # ax est un Axes Matplotlib
ax.set_title("Prix de cloture")
ax.set_ylabel("USD")
ax.grid(True, alpha=0.3)

fig = ax.get_figure()        # remonter à la Figure
fig.savefig("prix.png", dpi=150)
```

Pourquoi ça compte. Retenir que `ax = df.plot()` rend un Axes : Pandas trace vite, et l'on garde ensuite toute la puissance de Matplotlib pour le titre, les axes et l'export. Les deux outils se complètent, ils ne s'opposent pas.

3. Un graphique complet

Beaucoup de réglages passent directement en paramètres de .plot(), sans toucher à Matplotlib. On nomme les axes, on titre, on dimensionne, on choisit la grille — le tout en un seul appel.

```
ax = df.plot(
    y=["ES", "NQ"],          # colonnes à tracer
    title="Comparaison ES / NQ",
    figsize=(8, 4),
    grid=True,
    xlabel="Date", ylabel="Prix",
    linewidth=2,
)
ax.legend(title="Actif")     # ajustement Matplotlib
```

Paramètre	Rôle
x= / y=	Colonnes à mettre en abscisse / ordonnée.
title=	Titre du graphique.
figsize=	Taille de la figure, en pouces (largeur, hauteur).
grid=	Affiche la grille (True / False).

Paramètre	Rôle
xlabel= / ylabel=	Libellés des axes.
legend=	Affiche ou masque la légende (True / False).

La légende est gratuite. Contrairement à Matplotlib, pas besoin de label= : Pandas utilise le nom des colonnes. Tracer plusieurs colonnes nommées donne directement une légende correcte.

4. Les types de tracés

On choisit le type de deux façons équivalentes : le paramètre kind=, ou l'accessor .plot.<type>. La seconde forme est plus lisible et auto-documentée.

```
df["close"].plot(kind="hist", bins=30) # via kind=
df["close"].plot.hist(bins=30)        # équivalent, plus lisible
```

Accesseur	kind=	Quand l'utiliser
.plot.line	"line"	Évolution dans le temps (défaut).
.plot.bar	"bar"	Comparer des catégories (volumes).
.plot.hist	"hist"	Distribution d'une variable (rendements).
.plot.box	"box"	Dispersion et valeurs aberrantes.
.plot.scatter	"scatter"	Relation entre deux colonnes (x= et y=).
.plot.area	"area"	Aires empilées (composition d'un portefeuille).

Le nuage de points fait exception : il faut nommer explicitement les colonnes x= et y=, car Pandas ne peut pas deviner quelle colonne va sur quel axe.

```
df.plot.scatter(x="rendement_es", y="rendement_nq", alpha=0.5)
```

Pourquoi ça compte. Préférer la forme .plot.hist(...) à .plot(kind="hist"). Elle se lit comme une phrase, et l'éditeur de code propose les types disponibles par autocomplétion.

5. Plusieurs graphiques : subplots et ax=

5.1 subplots=True : une grille automatique

Pour séparer chaque colonne dans son propre graphique plutôt que de les superposer, il suffit de subplots=True. Pandas crée la grille et répartit les colonnes tout seul.

```
axes = df.plot(
    subplots=True,          # une colonne par sous-graphique
    layout=(2, 2),         # disposition de la grille
    figsize=(10, 6),
    sharex=True,           # axe x commun
)
```

subplots=True renvoie alors un tableau d'Axes (un par colonne), que l'on peut continuer à personnaliser individuellement, exactement comme avec plt.subplots du document 10.

5.2 ax= : superposer sur un graphique existant

À l'inverse, pour tracer plusieurs séries sur le même graphique — quand elles ne viennent pas du même DataFrame — on passe un même Axes via `ax=`.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8, 4))
es["close"].plot(ax=ax, label="ES")
nq["close"].plot(ax=ax, label="NQ")    # meme axes
ax.legend()
```

Pourquoi ça compte. `subplots=True` pour éclater les colonnes en une grille ; `ax=mon_axes` pour les superposer sur un graphique commun. Le paramètre `ax=` est le pont entre Pandas et une figure Matplotlib que l'on construit soi-même.

6. Styles et personnalisation

Comme `.plot()` s'appuie sur Matplotlib, toute la personnalisation du document 10 reste valable. Trois leviers se combinent.

En paramètres de `.plot()`. `color`, `style`, `linewidth`, `alpha`, `colormap`... passés directement à l'appel. Le plus rapide.

Sur l'Axes retourné. On récupère `ax = df.plot()` puis on applique `set_title`, `axhline`, `annotate`... (document 10).

Feuille de style globale. `plt.style.use("ggplot")` avant de tracer s'applique aussi aux graphiques Pandas, qui héritent du thème.

```
import matplotlib.pyplot as plt

plt.style.use("ggplot")

ax = df.plot(
    color={"ES": "steelblue", "NQ": "orange"},
    style=["-", "--"],          # un style de ligne par colonne
    alpha=0.8,
)
ax.axhline(0, color="grey", linewidth=0.8)
```

Paramètre	Effet
<code>color=</code>	Couleur(s) ; un dict {colonne: couleur} pour cibler.
<code>style=</code>	Style de ligne par colonne ("-", "--", ":").
<code>colormap=</code>	Palette pour de nombreuses colonnes ("viridis").
<code>alpha=</code>	Transparence, utile pour les nuages denses.
<code>secondary_y=</code>	Second axe y à droite (échelles différentes).

Pourquoi ça compte. Le réflexe : explorer avec les paramètres de `.plot()` pour aller vite, puis, dès que le rendu doit être soigné, récupérer l'Axes et finir avec Matplotlib. On ne choisit pas entre les deux, on enchaîne.

7. Les outils de `pandas.plotting`

Au-delà de `.plot()`, le sous-module `pandas.plotting` offre quelques tracés statistiques tout faits, pratiques en phase d'analyse exploratoire.

```
from pandas.plotting import scatter_matrix, autocorrelation_plot

# matrice de nuages : toutes les colonnes croisees deux a deux
scatter_matrix(df[["ES", "NQ", "CL"]], figsize=(8, 8), alpha=0.4)

# autocorrelation d'une serie de rendements
autocorrelation_plot(df["rendement_es"])
```

Fonction	Ce qu'elle montre
<code>scatter_matrix</code>	Toutes les paires de colonnes en nuages + histogrammes en diagonale.
<code>autocorrelation_plot</code>	Corrélation d'une série avec ses propres décalages.
<code>lag_plot</code>	Valeur à t en fonction de la valeur à $t-1$.
<code>bootstrap_plot</code>	Incertitude d'une statistique par rééchantillonnage.

Usage exploratoire. `scatter_matrix` donne en un coup d'œil la structure de corrélation d'un panier d'actifs. C'est un point de départ d'analyse, pas un graphique de rapport — pour ça, on construit une figure dédiée.

8. Sauvegarde et export

Pandas ne sauvegarde pas lui-même : l'export reste l'affaire de Matplotlib. On remonte de l'Axes à la Figure, puis on appelle `savefig` comme au document 10.

```
ax = df.plot(title="Prix de cloture", figsize=(8, 4))

fig = ax.get_figure()      # Axes -> Figure
fig.savefig("rapport.pdf")  # vectoriel, impression
fig.savefig("rapport.png", dpi=150,
            bbox_inches="tight") # ecran, marges rognees
```

Le seul réflexe propre à Pandas est ce `ax.get_figure()` : il fait le lien entre l'Axes renvoyé par `.plot()` et la Figure que `savefig` attend. Avec `subplots=True`, on passe par `axes[0].get_figure()` (les sous-graphiques partagent la même Figure).

Pourquoi ça compte. Tout ce qui concerne l'export — formats PNG/PDF/SVG, dpi, sauvegarder avant `show()` — a déjà été vu au document 10 et s'applique tel quel. Pandas s'arrête à `ax.get_figure()` ; Matplotlib prend le relais.

Récapitulatif

- 1. Le rôle.** Tracer en une ligne depuis un DataFrame, pour explorer vite. `.plot()` s'appuie sur Matplotlib.
- 2. L'index = l'axe x.** L'index (souvent une date) devient l'abscisse ; chaque colonne devient une courbe nommée.
- 3. `.plot()` renvoie un Axes.** On le récupère pour continuer avec Matplotlib (titre, axes, export).

4. Un graphique complet. `title=`, `figsize=`, `grid=`, `xlabel/ylabel=` en paramètres ; légende gratuite via le nom des colonnes.

5. Les types. `.plot.line/bar/hist/box/scatter/area`, ou `kind=`. Le scatter exige `x=` et `y=`.

6. Plusieurs graphiques. `subplots=True` pour éclater les colonnes ; `ax=` pour superposer sur un axes commun.

7. pandas.plotting. `scatter_matrix`, `autocorrelation_plot...` pour l'analyse exploratoire.

8. Export. `ax.get_figure()` puis `fig.savefig()` : l'export reste l'affaire de Matplotlib.

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Q1. Sur quoi repose la méthode `.plot()` de Pandas ?

Réponse. Sur Matplotlib, qu'elle appelle en coulisse. Pandas n'est qu'un raccourci pour tracer directement depuis un DataFrame ; le moteur de dessin reste Matplotlib.

Q2. Que devient l'index du DataFrame quand on appelle `df.plot()` ?

Réponse. Il devient l'axe x. Comme l'index est souvent une date (série temporelle), l'abscisse est correctement formatée sans aucun réglage — c'est l'atout principal de Pandas pour les prix dans le temps.

Q3. Pourquoi dit-on que `.plot()` « renvoie un Axes » est important ?

Réponse. Parce que `ax = df.plot()` rend un objet Axes Matplotlib : on garde ensuite toute la puissance de Matplotlib (`set_title`, `axhline`, `annotate`, `get_figure` pour l'export). Pandas trace vite, Matplotlib peaufine.

Q4. Comment tracer chaque colonne dans son propre graphique ?

Réponse. Avec `subplots=True` (et `layout=(lignes, colonnes)` pour la disposition). Pandas crée la grille et répartit les colonnes ; il renvoie alors un tableau d'Axes.

Q5. Quelle particularité a le tracé en nuage de points (scatter) ?

Réponse. Il faut nommer explicitement `x=` et `y=` : `df.plot.scatter(x="a", y="b")`. Contrairement aux autres types, Pandas ne peut pas deviner quelle colonne va sur quel axe.

Q6. Comment sauvegarder un graphique Pandas en fichier ?

Réponse. Pandas ne sauvegarde pas : on remonte de l'Axes à la Figure avec `fig = ax.get_figure()`, puis `fig.savefig("...")`. L'export reste entièrement du ressort de Matplotlib (document 10).

Document suivant (30) : *Seaborn — graphiques statistiques élégants (distributions, corrélations, catégories) bâtis sur Matplotlib, avec un style soigné par défaut.*